Dynamic Fire Evacuation Router with Real-Time Hazard Mapping

Source Code Appendix

Submitted by: Kishaunjith S

Date: July 27, 2026

This appendix contains the core source files for the Dynamic Fire Evacuation Router project, provided as a readable PDF listing since the submission portal does not accept raw script files inside a zip archive. The complete, runnable repository, including full commit history and all supporting documentation, is publicly available at: github.com/kishaunjith-S/fire-evac-router.

node/graph_config.py

Building graph: 6x6 zone grid, exit topology, JSON export for the dashboard.

```
"""Building graph for the fire evacuation router.

6x6 grid of zones. Zone ids are "r{row}c{col}", 0-indexed. Two exits sit at
opposite corners. Edge weight defaults to 1 and is later overwritten at
runtime from live hazard costs per docs/PROJECT_SPEC.md
(w(u, v) = (zone_cost(u) + zone_cost(v)) / 2).
"""

import json

import networkx as nx

GRID_ROWS = 6
GRID_COLS = 6

EXIT_ZONES = ["r0c0", "r5c5"]

DEFAULT_EDGE_WEIGHT = 1


def zone_id(row, col):
    return f"r{row}c{col}"


def build_graph():
    """Build the 6x6 zone grid graph with default edge weights."""
    graph = nx.Graph()

    for row in range(GRID_ROWS):
        for col in range(GRID_COLS):
            graph.add_node(
                zone_id(row, col),
                row=row,
                col=col,
                is_exit=zone_id(row, col) in EXIT_ZONES,
            )

    for row in range(GRID_ROWS):
        for col in range(GRID_COLS):
            here = zone_id(row, col)
            if col + 1 < GRID_COLS:
                graph.add_edge(here, zone_id(row, col + 1), weight=DEFAULT_EDGE_WEIGHT)
            if row + 1 < GRID_ROWS:
                graph.add_edge(here, zone_id(row + 1, col), weight=DEFAULT_EDGE_WEIGHT)

    return graph


def graph_to_json(graph):
    """Export the graph as a JSON-serializable dict for the dashboard.

    Shape:
        {
          "nodes": [{"id": str, "row": int, "col": int, "is_exit": bool}, ...],
          "edges": [{"source": str, "target": str, "weight": float}, ...],
          "exits": [str, ...]
        }
    """
    nodes = [
        {
            "id": node,
            "row": data["row"],
            "col": data["col"],
            "is_exit": data["is_exit"],
        }
        for node, data in graph.nodes(data=True)
    ]

    edges = [
```

```
        {"source": u, "target": v, "weight": data.get("weight", DEFAULT_EDGE_WEIGHT)}
        for u, v, data in graph.edges(data=True)
    ]

    return {"nodes": nodes, "edges": edges, "exits": EXIT_ZONES}


def export_graph_json(path=None):
    """Build the graph and export it as JSON, returning the JSON string.

    If `path` is given, also writes the JSON to that file.
    """
    payload = graph_to_json(build_graph())
    text = json.dumps(payload, indent=2)

    if path is not None:
        with open(path, "w") as f:
            f.write(text)

    return text


if __name__ == "__main__":
    print(export_graph_json())
```

node/routing_common.py

Shared pure logic: sensor fusion formula, edge weighting, and the single multi-source Dijkstra routing computation used by the coordinator.

```
"""Shared, stateless routing helpers used by both fire_node.py (per-zone
fusion + local hysteresis) and routing_coordinator.py (the single global
Dijkstra solve). Keeping the formulas here means both processes agree on
one definition instead of maintaining drifting copies.
"""

import networkx as nx

# --- Sensor fusion constants (docs/PROJECT_SPEC.md: Sensor fusion formula) ---
ALPHA = 0.05
BETA = 0.002
AMBIENT_C = 22.0
FLAME_PENALTY = 1000
BASELINE_COST = 1.0

# Color thresholds on zone_cost. Tunable, same spirit as alpha/beta/FLAME_PENALTY.
YELLOW_COST_THRESHOLD = 5.0
RED_COST_THRESHOLD = 50.0

EXIT_SINK = "__EXIT__"

# A zone at or above this cost is a hard block, not just expensive: edges
# touching it are removed from the routing graph entirely. Without this, an
# isolated zone never actually reaches float("inf") -- FLAME_PENALTY is only
# a very large weight, and a route through it, however absurd, always exists
# on an unchanged topology. Removing the edge is what makes shelter-in-place
# ("no path to any exit") a real, reachable state instead of a dead branch.
BLOCKED_COST_THRESHOLD = FLAME_PENALTY


def zone_cost(temp, smoke, flame):
    """Fusion formula: raw sensor readings -> per-zone hazard cost."""
    return (
        BASELINE_COST
        + ALPHA * max(0.0, temp - AMBIENT_C)
        + BETA * (smoke ** 1.5)
        + (FLAME_PENALTY if flame else 0.0)
    )


def edge_weight(cost_u, cost_v):
    """Zone cost -> edge weight: average of both endpoints."""
    return (cost_u + cost_v) / 2.0


def color_for_cost(cost, flame):
    if flame or cost > RED_COST_THRESHOLD:
        return "red"
    if cost > YELLOW_COST_THRESHOLD:
        return "yellow"
    return "green"


def direction_between(graph, from_zone, to_zone):
    """Compass direction to move from from_zone to the adjacent to_zone."""
    from_row, from_col = graph.nodes[from_zone]["row"], graph.nodes[from_zone]["col"]
    to_row, to_col = graph.nodes[to_zone]["row"], graph.nodes[to_zone]["col"]
    if to_row < from_row:
        return "N"
    if to_row > from_row:
        return "S"
    if to_col > from_col:
        return "E"
    if to_col < from_col:
        return "W"
    return None

def build_weighted_graph_with_sink(base_graph, hazard_costs, exit_zones):
    """base_graph's edges plus a virtual sink wired to the exits, with
```

```
    hard-blocked edges removed (see BLOCKED_COST_THRESHOLD)."""
    g = nx.Graph()
    g.add_nodes_from(base_graph.nodes)
    for u, v in base_graph.edges:
        if hazard_costs[u] >= BLOCKED_COST_THRESHOLD or hazard_costs[v] >=
            BLOCKED_COST_THRESHOLD:
            continue  # hard block: edge removed, not just expensive
        g.add_edge(u, v, weight=edge_weight(hazard_costs[u], hazard_costs[v]))

    g.add_node(EXIT_SINK)
    for exit_zone in exit_zones:
        g.add_edge(EXIT_SINK, exit_zone, weight=0.0)

    return g


def compute_routing(base_graph, hazard_costs, exit_zones):
    """Single reverse multi-source Dijkstra from the exits.

    Returns (dist_to_exit, next_hop) dicts covering every real zone in
    base_graph. dist_to_exit[z] is float("inf") and next_hop[z] is None for
    zones with no route to any exit (shelter-in-place). next_hop[z] is also
    None for exit zones themselves (already arrived).
    """
    g = build_weighted_graph_with_sink(base_graph, hazard_costs, exit_zones)
    lengths, paths = nx.single_source_dijkstra(g, source=EXIT_SINK)

    dist_to_exit = {}
    next_hop = {}
    for z in base_graph.nodes:
        if z not in lengths:
            dist_to_exit[z] = float("inf")
            next_hop[z] = None
            continue
        dist_to_exit[z] = lengths[z]
        path = paths[z]  # [EXIT_SINK, ..., z]
        next_hop[z] = path[-2] if len(path) > 1 and path[-2] != EXIT_SINK else None

    return dist_to_exit, next_hop
```

node/routing_coordinator.py

Single centralized coordinator: one multi-source Dijkstra solve per hazard update, broadcast to all zone nodes. Fixes the O(N^2) fan-out bug described in docs/latency_debugging_notes.md.

```
"""Routing coordinator -- solves the global multi-source Dijkstra once
per hazard update and broadcasts the full routing field. Centralizes what
was previously an O(zones^2) per-node recompute fan-out into a single
O(zones) recompute per hazard message (see docs/latency_debugging_notes.md).

Run:
    python routing_coordinator.py
"""

import argparse
import json
import logging
import threading
import time

import paho.mqtt.client as mqtt

import graph_config
import routing_common as rc

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(name)s %(message)s")

TOPIC_HAZARD_WILDCARD = "fire/nodes/+/hazard"
TOPIC_ROUTING = "fire/system/routing"
TOPIC_HEALTH = "fire/system/health/coordinator"
MAX_QUEUED_MESSAGES = 100


class RoutingCoordinator:
    def __init__(self, broker_host="localhost", broker_port=1883):
        self.broker_host = broker_host
        self.broker_port = broker_port
        self.graph = graph_config.build_graph()
        self.exit_zones = graph_config.EXIT_ZONES
        self.log = logging.getLogger("routing_coordinator")
        self._lock = threading.Lock()
        self._hazard_costs = {z: rc.BASELINE_COST for z in self.graph.nodes}
        self._recompute_count = 0

        self._client = mqtt.Client(client_id="routing_coordinator")
        self._client.max_queued_messages_set(MAX_QUEUED_MESSAGES)
        will_payload = json.dumps({"state": "offline", "ts": time.time()})
        self._client.will_set(TOPIC_HEALTH, will_payload, qos=0, retain=True)
        self._client.on_connect = self._on_connect
        self._client.on_message = self._on_message

    def start(self):
        self._client.connect(self.broker_host, self.broker_port)
        self._client.loop_start()

    def stop(self):
        self._client.loop_stop()
        self._client.disconnect()

    def _on_connect(self, client, userdata, flags, rc_code):
        self.log.info("connected to broker (rc=%s)", rc_code)
        client.subscribe(TOPIC_HAZARD_WILDCARD, qos=1)
        self._publish_health("ok")
        self._recompute_and_publish(src_ts=time.time())

    def _on_message(self, client, userdata, msg):
        try:
            payload = json.loads(msg.payload.decode("utf-8"))
        except (ValueError, UnicodeDecodeError):
            self.log.warning("discarding malformed payload on %s", msg.topic)
            return
        zone_id = msg.topic.split("/")[2]
```

```
        if zone_id not in self._hazard_costs:
            return
        try:
            cost = float(payload["cost"])
        except (KeyError, TypeError, ValueError):
            self.log.warning("discarding malformed hazard payload: %r", payload)
            return
        with self._lock:
            self._hazard_costs[zone_id] = cost
        self._recompute_and_publish(src_ts=payload.get("ts", time.time()))

    def _recompute_and_publish(self, src_ts):
        self._recompute_count += 1
        with self._lock:
            hazard_costs = dict(self._hazard_costs)
        dist_to_exit, next_hop = rc.compute_routing(self.graph, hazard_costs, self.exit_zones)
        payload = json.dumps({
            "hazard_costs": hazard_costs,
            "dist_to_exit": {z: (d if d != float("inf") else None) for z, d in
                dist_to_exit.items()},
            "next_hop": next_hop,
            "src_ts": src_ts,
            "ts": time.time(),
        })
        self._client.publish(TOPIC_ROUTING, payload, qos=1, retain=True)

    def _publish_health(self, state):
        payload = json.dumps({"state": state, "ts": time.time()})
        self._client.publish(TOPIC_HEALTH, payload, qos=0, retain=True)


def main():
    parser = argparse.ArgumentParser(description="Global routing coordinator")
    parser.add_argument("--broker-host", default="localhost")
    parser.add_argument("--broker-port", type=int, default=1883)
    args = parser.parse_args()
    coordinator = RoutingCoordinator(broker_host=args.broker_host, broker_port=args.broker_port)
    coordinator.start()
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        coordinator.stop()


if __name__ == "__main__":
    main()
```

node/fire_node.py

Per-zone node: sensor ingestion, hazard publish, hysteresis-gated LED decision, fail-safe watchdog. One process per building zone.

```
"""Fire Node -- simulates one zone's MCU. One process per zone: reads
its own sensor topic, publishes hazard cost, subscribes to the
coordinator's routing broadcast, applies local hysteresis + LED logic,
and runs a fail-safe watchdog for sensor silence.

Run one instance per zone:
    python fire_node.py --zone r0c0
"""

import argparse
import json
import logging
import threading
import time

import paho.mqtt.client as mqtt

import graph_config
import routing_common as rc

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(name)s %(message)s")

HYSTERESIS_MARGIN = 0.05
MIN_DWELL_MS = 500
LED_KEEPALIVE_S = 5.0
MAX_QUEUED_MESSAGES = 100
SENSOR_TIMEOUT_S = 5.0
HEALTH_PUBLISH_INTERVAL_S = 2.0
WATCHDOG_INTERVAL_S = 1.0

TOPIC_SENSOR = "fire/sensors/{zone_id}"
TOPIC_HAZARD = "fire/nodes/{zone_id}/hazard"
TOPIC_ROUTING = "fire/system/routing"
TOPIC_LED = "fire/nodes/{zone_id}/led"
TOPIC_HEALTH = "fire/system/health/{zone_id}"


class FireNode:
    def __init__(self, zone_id, broker_host="localhost", broker_port=1883):
        if zone_id not in graph_config.build_graph().nodes:
            raise ValueError(f"unknown zone_id: {zone_id}")
        self.zone_id = zone_id
        self.broker_host = broker_host
        self.broker_port = broker_port
        self.graph = graph_config.build_graph()
        self.is_exit = self.graph.nodes[zone_id]["is_exit"]
        self.log = logging.getLogger(f"fire_node[{zone_id}]")
        self._own_cost = rc.BASELINE_COST
        self._own_flame = False
        self._last_sensor_ts = None
        self._displayed_next_hop = None
        self._displayed_route_cost = None
        self._last_change_ts = 0.0
        self._last_published_led = None
        self._last_led_publish_ts = 0.0
        self._degraded = False
        self._stopping = False
        self._led_publish_count = 0
        self._routing_messages_received = 0

        self._client = mqtt.Client(client_id=f"fire_node_{zone_id}")
        self._client.max_queued_messages_set(MAX_QUEUED_MESSAGES)
        will_payload = json.dumps({"state": "offline", "ts": time.time()})
        self._client.will_set(TOPIC_HEALTH.format(zone_id=zone_id), will_payload, qos=0,
            retain=True)
        self._client.on_connect = self._on_connect
        self._client.on_message = self._on_message

    def start(self):
```

```
    self._client.connect(self.broker_host, self.broker_port)
    self._client.loop_start()
    self._watchdog_thread = threading.Thread(target=self._watchdog_loop, daemon=True)
    self._watchdog_thread.start()

def stop(self):
    self._stopping = True
    self._client.loop_stop()
    self._client.disconnect()

def _on_connect(self, client, userdata, flags, rc_code):
    self.log.info("connected to broker (rc=%s)", rc_code)
    client.subscribe(TOPIC_SENSOR.format(zone_id=self.zone_id), qos=1)
    client.subscribe(TOPIC_ROUTING, qos=1)
    self._publish_health("ok")

def _on_message(self, client, userdata, msg):
    try:
        payload = json.loads(msg.payload.decode("utf-8"))
    except (ValueError, UnicodeDecodeError):
        self.log.warning("discarding malformed payload on %s", msg.topic)
        return
    if msg.topic == TOPIC_SENSOR.format(zone_id=self.zone_id):
        self._handle_sensor_reading(payload)
    elif msg.topic == TOPIC_ROUTING:
        self._handle_routing_update(payload)

def _handle_sensor_reading(self, payload):
    try:
        temp = float(payload["temp"])
        smoke = float(payload["smoke"])
        flame = bool(payload["flame"])
    except (KeyError, TypeError, ValueError):
        self.log.warning("discarding malformed sensor payload: %r", payload)
        return
    self._last_sensor_ts = time.time()
    if self._degraded:
        self._degraded = False
        self._publish_health("ok")
    self._own_flame = flame
    self._own_cost = rc.zone_cost(temp, smoke, flame)
    self._publish_hazard(self._own_cost, payload.get("ts", time.time()))

def _handle_routing_update(self, payload):
    try:
        hazard_costs = payload["hazard_costs"]
        dist_to_exit = payload["dist_to_exit"]
        next_hop_map = payload["next_hop"]
        src_ts = payload["src_ts"]
    except (KeyError, TypeError):
        self.log.warning("discarding malformed routing payload: %r", payload)
        return
    self._routing_messages_received += 1
    self._apply_routing(hazard_costs, dist_to_exit, next_hop_map, src_ts)

def _apply_routing(self, hazard_costs, dist_to_exit, next_hop_map, src_ts):
    own_dist = dist_to_exit.get(self.zone_id)
    unreachable = own_dist is None
    at_exit = self.is_exit
    if unreachable:
        self._displayed_next_hop = None
        self._displayed_route_cost = float("inf")
        self._last_change_ts = time.time()
        direction = None
        state = "shelter_in_place"
        color = "red"
    elif at_exit:
        self._displayed_next_hop = None
        self._displayed_route_cost = 0.0
        direction = None
        state = "normal"
        color = rc.color_for_cost(self._own_cost, flame=self._own_flame)
    else:
        candidate_hop = next_hop_map.get(self.zone_id)
        candidate_cost = own_dist
```

```
            chosen_hop = self._apply_hysteresis(candidate_hop, candidate_cost, hazard_costs,
                dist_to_exit)
            direction = rc.direction_between(self.graph, self.zone_id, chosen_hop)
            state = "normal"
            color = rc.color_for_cost(self._own_cost, flame=self._own_flame)
        self._publish_led_if_needed(color, direction, state, src_ts)

    def _publish_led_if_needed(self, color, direction, state, src_ts):
        current = (color, direction, state)
        now = time.time()
        changed = current != self._last_published_led
        keepalive_due = (now - self._last_led_publish_ts) >= LED_KEEPALIVE_S
        if not (changed or keepalive_due):
            return
        self._publish_led(color, direction, state, src_ts)
        self._last_published_led = current
        self._last_led_publish_ts = now

    def _apply_hysteresis(self, candidate_hop, candidate_cost, neighbor_costs, dist_to_exit):
        now = time.time()
        if self._displayed_next_hop is None or self._displayed_route_cost is None:
            self._displayed_next_hop = candidate_hop
            self._displayed_route_cost = candidate_cost
            self._last_change_ts = now
            return candidate_hop
        if self._own_flame:
            if candidate_hop != self._displayed_next_hop:
                self._displayed_next_hop = candidate_hop
                self._displayed_route_cost = candidate_cost
                self._last_change_ts = now
            return candidate_hop
        if candidate_hop == self._displayed_next_hop:
            self._displayed_route_cost = candidate_cost
            return self._displayed_next_hop
        current_hop = self._displayed_next_hop
        hop_cost = neighbor_costs.get(current_hop, float("inf"))
        if self._own_cost >= rc.BLOCKED_COST_THRESHOLD or hop_cost >= rc.BLOCKED_COST_THRESHOLD:
            current_route_cost = float("inf")
        else:
            hop_dist = dist_to_exit.get(current_hop)
            hop_dist = float("inf") if hop_dist is None else hop_dist
            current_route_cost = rc.edge_weight(self._own_cost, hop_cost) + hop_dist
        if current_route_cost == float("inf"):
            self._displayed_next_hop = candidate_hop
            self._displayed_route_cost = candidate_cost
            self._last_change_ts = now
            return candidate_hop
        dwell_elapsed_ms = (now - self._last_change_ts) * 1000.0
        cheap_enough = candidate_cost < current_route_cost * (1 - HYSTERESIS_MARGIN)
        if cheap_enough and dwell_elapsed_ms >= MIN_DWELL_MS:
            self._displayed_next_hop = candidate_hop
            self._displayed_route_cost = candidate_cost
            self._last_change_ts = now
            return candidate_hop
        self._displayed_route_cost = current_route_cost
        return current_hop

    def _publish_hazard(self, cost, ts):
        payload = json.dumps({"cost": cost, "ts": ts})
        self._client.publish(TOPIC_HAZARD.format(zone_id=self.zone_id), payload, qos=1,
            retain=True)

    def _publish_led(self, color, direction, state, src_ts):
        self._led_publish_count += 1
        payload = json.dumps({
            "color": color, "direction": direction, "state": state,
            "src_ts": src_ts, "ts": time.time(),
        })
        self._client.publish(TOPIC_LED.format(zone_id=self.zone_id), payload, qos=1,
            retain=True)

    def _publish_health(self, state):
        payload = json.dumps({"state": state, "ts": time.time()})
        self._client.publish(TOPIC_HEALTH.format(zone_id=self.zone_id), payload, qos=0,
            retain=True)
```

```
    def _watchdog_loop(self):
        last_health_publish = 0.0
        while not self._stopping:
            time.sleep(WATCHDOG_INTERVAL_S)
            now = time.time()
            if self._last_sensor_ts is not None:
                silent_for = now - self._last_sensor_ts
                if silent_for > SENSOR_TIMEOUT_S and not self._degraded:
                    self._degraded = True
                    self.log.warning("no sensor update for %.1fs, falling back to
                        last-known-safe path", silent_for)
                    self._publish_health("degraded")
            if now - last_health_publish >= HEALTH_PUBLISH_INTERVAL_S:
                self._publish_health("degraded" if self._degraded else "ok")
                last_health_publish = now


def main():
    parser = argparse.ArgumentParser(description="Fire node MCU simulator")
    parser.add_argument("--zone", required=True, help="zone id, e.g. r0c0")
    parser.add_argument("--broker-host", default="localhost")
    parser.add_argument("--broker-port", type=int, default=1883)
    args = parser.parse_args()
    node = FireNode(args.zone, broker_host=args.broker_host, broker_port=args.broker_port)
    node.start()
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        node.stop()


if __name__ == "__main__":
    main()
```

node/run_all_nodes.py

Single-command demo runner: starts the coordinator and all 36 fire nodes together, plus temporary diagnostic instrumentation used during load testing.

```
"""Runs the routing coordinator plus all 36 zone nodes in one process.
Each zone still gets its own FireNode instance with its own MQTT client
and watchdog thread; only the Dijkstra solve is centralized in one
RoutingCoordinator instance.

Run:
    python run_all_nodes.py
"""

import argparse
import logging
import sys
import threading
import time

import graph_config
from fire_node import FireNode
from routing_coordinator import RoutingCoordinator

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(message)s")
log = logging.getLogger("run_all_nodes")
diag_log = logging.getLogger("diagnostic")

STAGGER_S = 0.02
DIAG_INTERVAL_S = 10.0


def _diagnostic_loop(coordinator, nodes, stop_event):
    prev_led_counts = {n.zone_id: 0 for n in nodes}
    prev_recompute_count = 0
    while not stop_event.is_set():
        stop_event.wait(DIAG_INTERVAL_S)
        if stop_event.is_set():
            break
        thread_count = threading.active_count()
        out_messages_total = sum(len(n._client._out_messages) for n in nodes)
        out_messages_total += len(coordinator._client._out_messages)
        out_packet_total = sum(len(n._client._out_packet) for n in nodes)
        out_packet_total += len(coordinator._client._out_packet)
        led_counts = {n.zone_id: n._led_publish_count for n in nodes}
        total_led_publishes = sum(led_counts.values())
        led_delta = total_led_publishes - sum(prev_led_counts.values())
        prev_led_counts = led_counts
        recompute_delta = coordinator._recompute_count - prev_recompute_count
        prev_recompute_count = coordinator._recompute_count
        diag_log.info(
            "threads=%d out_messages=%d out_packet=%d recomputes/%ds=%d led_publishes/%ds=%d
                (ratio=%.1f)",
            thread_count, out_messages_total, out_packet_total,
            int(DIAG_INTERVAL_S), recompute_delta, int(DIAG_INTERVAL_S), led_delta,
            (led_delta / recompute_delta) if recompute_delta else 0.0,
        )


def main():
    parser = argparse.ArgumentParser(description="Run coordinator and all zone fire nodes in
        one process")
    parser.add_argument("--broker-host", default="localhost")
    parser.add_argument("--broker-port", type=int, default=1883)
    args = parser.parse_args()
    zone_ids = list(graph_config.build_graph().nodes)
    coordinator = RoutingCoordinator(broker_host=args.broker_host, broker_port=args.broker_port)
    nodes = [FireNode(z, broker_host=args.broker_host, broker_port=args.broker_port) for z in
        zone_ids]
    log.info("starting routing coordinator + %d fire nodes against %s:%s", len(nodes),
        args.broker_host, args.broker_port)
    started = []
    try:
```

```
        coordinator.start()
        for node in nodes:
            node.start()
            started.append(node)
            time.sleep(STAGGER_S)
    except Exception:
        log.exception("failed to start (is the Mosquitto broker running?) -- stopping %d
            node(s)", len(started))
        for node in started:
            node.stop()
        coordinator.stop()
        sys.exit(1)
    log.info("routing coordinator + all %d fire nodes running -- press Ctrl+C to stop",
        len(nodes))
    diag_stop_event = threading.Event()
    diag_thread = threading.Thread(target=_diagnostic_loop, args=(coordinator, nodes,
        diag_stop_event), daemon=True)
    diag_thread.start()
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        pass
    finally:
        diag_stop_event.set()
        log.info("stopping all fire nodes and the routing coordinator")
        for node in nodes:
            node.stop()
        coordinator.stop()


if __name__ == "__main__":
    main()
```

injector/app.py

Simulation/injection tool: Flask control-panel backend, ambient sensor ticker, flashover trigger endpoint, and live state streaming to the UI.

```
"""Injector -- Flask control panel that simulates fire sensors.
Publishes raw sensor payloads to fire/sensors/{zone_id}. Also subscribes
to the nodes' hazard/LED/health topics so the browser panel can render
live state over Server-Sent Events.

Run:
    python app.py
"""

import json
import os
import queue
import random
import sys
import threading
import time

from flask import Flask, Response, jsonify, render_template, request
import paho.mqtt.client as mqtt

sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "node"))
import graph_config  # noqa: E402

MQTT_BROKER_HOST = os.environ.get("MQTT_BROKER_HOST", "localhost")
MQTT_BROKER_PORT = int(os.environ.get("MQTT_BROKER_PORT", "1883"))

TOPIC_SENSOR = "fire/sensors/{zone_id}"
TOPIC_HAZARD_WILDCARD = "fire/nodes/+/hazard"
TOPIC_LED_WILDCARD = "fire/nodes/+/led"
TOPIC_HEALTH_WILDCARD = "fire/system/health/+"

AMBIENT_TICK_S = 1.0
AMBIENT_TEMP_RANGE = (20.0, 24.0)
AMBIENT_SMOKE_RANGE = (0.0, 5.0)
FLASHOVER_TEMP_RANGE = (550.0, 650.0)
FLASHOVER_SMOKE_RANGE = (2500.0, 3500.0)

app = Flask(__name__)
_graph = graph_config.build_graph()
_zone_ids = list(_graph.nodes)
_state_lock = threading.Lock()
_zone_state = {
    z: {"zone_id": z, "cost": None, "color": None, "direction": None,
        "state": None, "health": "unknown", "last_sensor": None}
    for z in _zone_ids
}
_active_flashovers = set()
_subscribers = []
_subscribers_lock = threading.Lock()


def _broadcast(event):
    payload = json.dumps(event)
    with _subscribers_lock:
        for q in _subscribers:
            q.put(payload)


def _on_connect(client, userdata, flags, rc):
    client.subscribe(TOPIC_HAZARD_WILDCARD, qos=1)
    client.subscribe(TOPIC_LED_WILDCARD, qos=1)
    client.subscribe(TOPIC_HEALTH_WILDCARD, qos=0)


def _on_message(client, userdata, msg):
    try:
        payload = json.loads(msg.payload.decode("utf-8"))
    except (ValueError, UnicodeDecodeError):
```

```
        return
    parts = msg.topic.split("/")
    event = None
    if msg.topic.startswith("fire/nodes/") and msg.topic.endswith("/hazard"):
        zone_id = parts[2]
        if zone_id not in _zone_state:
            return
        with _state_lock:
            _zone_state[zone_id]["cost"] = payload.get("cost")
        event = {"type": "hazard", "zone_id": zone_id, **payload}
    elif msg.topic.startswith("fire/nodes/") and msg.topic.endswith("/led"):
        zone_id = parts[2]
        if zone_id not in _zone_state:
            return
        with _state_lock:
            _zone_state[zone_id]["color"] = payload.get("color")
            _zone_state[zone_id]["direction"] = payload.get("direction")
            _zone_state[zone_id]["state"] = payload.get("state")
        event = {"type": "led", "zone_id": zone_id, **payload}
    elif msg.topic.startswith("fire/system/health/"):
        zone_id = parts[3]
        if zone_id not in _zone_state:
            return
        with _state_lock:
            _zone_state[zone_id]["health"] = payload.get("state")
        event = {"type": "health", "zone_id": zone_id, **payload}
    if event is not None:
        _broadcast(event)


_client = mqtt.Client(client_id="injector_app")
_client.on_connect = _on_connect
_client.on_message = _on_message


def _publish_sensor_reading(zone_id, temp, smoke, flame):
    ts = time.time()
    payload = json.dumps({"temp": temp, "smoke": smoke, "flame": flame, "ts": ts})
    _client.publish(TOPIC_SENSOR.format(zone_id=zone_id), payload, qos=1)
    with _state_lock:
        _zone_state[zone_id]["last_sensor"] = {"temp": temp, "smoke": smoke, "flame": flame,
            "ts": ts}
    _broadcast({"type": "sensor", "zone_id": zone_id, "temp": temp, "smoke": smoke, "flame":
        flame, "ts": ts})


def _ambient_reading():
    return (random.uniform(*AMBIENT_TEMP_RANGE), random.uniform(*AMBIENT_SMOKE_RANGE), False)


def _flashover_reading():
    return (random.uniform(*FLASHOVER_TEMP_RANGE), random.uniform(*FLASHOVER_SMOKE_RANGE), True)


def _ambient_ticker():
    while True:
        time.sleep(AMBIENT_TICK_S)
        with _state_lock:
            flashing = set(_active_flashovers)
        for zone_id in _zone_ids:
            if zone_id in flashing:
                temp, smoke, flame = _flashover_reading()
            else:
                temp, smoke, flame = _ambient_reading()
            _publish_sensor_reading(zone_id, temp, smoke, flame)


@app.route("/")
def index():
    return render_template("index.html")


@app.route("/api/graph")
def api_graph():
    return jsonify(graph_config.graph_to_json(_graph))
```

```
@app.route("/api/state")
def api_state():
    with _state_lock:
        zones = {z: dict(s) for z, s in _zone_state.items()}
        flashovers = list(_active_flashovers)
    return jsonify({"zones": zones, "active_flashovers": flashovers})


@app.route("/api/trigger", methods=["POST"])
def api_trigger():
    body = request.get_json(silent=True) or {}
    zone_id = body.get("zone_id")
    action = body.get("action")
    if zone_id not in _zone_state:
        return jsonify({"ok": False, "error": "unknown zone_id"}), 400
    if action not in ("flashover", "clear"):
        return jsonify({"ok": False, "error": "action must be flashover or clear"}), 400
    if action == "flashover":
        with _state_lock:
            _active_flashovers.add(zone_id)
        temp, smoke, flame = _flashover_reading()
    else:
        with _state_lock:
            _active_flashovers.discard(zone_id)
        temp, smoke, flame = _ambient_reading()
    _publish_sensor_reading(zone_id, temp, smoke, flame)
    return jsonify({"ok": True})


@app.route("/api/events")
def api_events():
    client_queue = queue.Queue()
    with _subscribers_lock:
        _subscribers.append(client_queue)

    def stream():
        try:
            while True:
                payload = client_queue.get()
                yield "data: " + payload + "\n\n"
        finally:
            with _subscribers_lock:
                if client_queue in _subscribers:
                    _subscribers.remove(client_queue)

    return Response(stream(), mimetype="text/event-stream")


def _start_background_threads():
    _client.connect(MQTT_BROKER_HOST, MQTT_BROKER_PORT)
    _client.loop_start()
    threading.Thread(target=_ambient_ticker, daemon=True).start()


if __name__ == "__main__":
    _start_background_threads()
    app.run(host="0.0.0.0", port=5000, debug=False, threaded=True, use_reloader=False)
```

firmware_wokwi/sketch.ino

Arduino C++ firmware port of the sensor-fusion formula, validated running on a simulated ESP32 in the Wokwi environment.

```
/*
 * Fire Evacuation Router -- Wokwi ESP32 firmware port.
 * Demonstrates the same sensor-fusion formula used in the full MQTT-based
 * system (node/fire_node.py + node/routing_common.py), ported to run on
 * real/simulated hardware. Does not implement MQTT, multi-node routing,
 * or Dijkstra pathfinding -- single-zone fusion + LED logic only.
 *
 * Wiring:
 *   DHT22 data        -> GPIO 4   (temperature)
 *   Potentiometer SIG  -> GPIO 34 (analog, stand-in for smoke ppm sensor)
 *   Pushbutton         -> GPIO 5, INPUT_PULLUP (stand-in for flame sensor)
 *   RGB LED            -> R: GPIO 25, G: GPIO 26, B: GPIO 27 (common anode)
 */

#include <DHT.h>

#define DHT_PIN 4
#define DHT_TYPE DHT22
#define SMOKE_PIN 34
#define FLAME_BUTTON_PIN 5
#define LED_R_PIN 25
#define LED_G_PIN 26
#define LED_B_PIN 27

const float ALPHA = 0.05;
const float BETA = 0.002;
const float AMBIENT_C = 22.0;
const float FLAME_PENALTY = 1000.0;
const float YELLOW_COST_THRESHOLD = 5.0;
const float RED_COST_THRESHOLD = 50.0;
const int ANALOG_MAX = 4095;
const float SMOKE_PPM_MAX = 500.0;
const unsigned long LOOP_INTERVAL_MS = 1000;

DHT dht(DHT_PIN, DHT_TYPE);

void setLedColor(bool red, bool green, bool blue) {
  digitalWrite(LED_R_PIN, red ? LOW : HIGH);
  digitalWrite(LED_G_PIN, green ? LOW : HIGH);
  digitalWrite(LED_B_PIN, blue ? LOW : HIGH);
}

float zoneCost(float tempC, float smokePpm, bool flame) {
  float cost = 1.0;
  cost += ALPHA * max(0.0f, tempC - AMBIENT_C);
  cost += BETA * pow(smokePpm, 1.5);
  if (flame) {
    cost += FLAME_PENALTY;
  }
  return cost;
}

void setup() {
  Serial.begin(115200);
  dht.begin();
  pinMode(FLAME_BUTTON_PIN, INPUT_PULLUP);
  pinMode(LED_R_PIN, OUTPUT);
  pinMode(LED_G_PIN, OUTPUT);
  pinMode(LED_B_PIN, OUTPUT);
  setLedColor(false, false, false);
}

void loop() {
  float tempC = dht.readTemperature();
  if (isnan(tempC)) {
    Serial.println("DHT22 read failed, skipping this cycle");
    delay(LOOP_INTERVAL_MS);
    return;
```

```
  }
  int rawSmoke = analogRead(SMOKE_PIN);
  float smokePpm = (rawSmoke / (float)ANALOG_MAX) * SMOKE_PPM_MAX;
  bool flame = (digitalRead(FLAME_BUTTON_PIN) == LOW);
  float cost = zoneCost(tempC, smokePpm, flame);

  if (flame || cost >= RED_COST_THRESHOLD) {
    setLedColor(true, false, false);
    Serial.println("SHELTER IN PLACE");
  } else if (cost >= YELLOW_COST_THRESHOLD) {
    setLedColor(true, true, false);
  } else {
    setLedColor(false, true, false);
  }

  Serial.print("temp=");
  Serial.print(tempC);
  Serial.print(" smoke_ppm=");
  Serial.print(smokePpm);
  Serial.print(" flame=");
  Serial.print(flame ? "true" : "false");
  Serial.print(" cost=");
  Serial.println(cost);

  delay(LOOP_INTERVAL_MS);
}
```